WebRTC NAT Traversal

Deep-Dive: Why NAT Poses a Problem for Peer-to-Peer Networks

Detailed Technical Report

MeetFlow Engineering Team

June 24, 2026

Contents

1 Introduction

The primary architectural goal of WebRTC (Web Real-Time Communication) is to establish a direct, **Peer-to-Peer (P2P)** connection between two web browsers. This design enables the exchange of audio, video, and arbitrary data channels with minimal latency, bypasses central server scaling limitations, and guarantees end-to-end encryption.

However, in the real-world Internet, establishing a direct connection between two arbitrary nodes is challenging. The main obstacle is **Network Address Translation (NAT)**. Designed initially as a temporary solution to mitigate IPv4 address exhaustion, NAT has become a ubiquitous standard in modern home routers, enterprise firewalls, and cellular networks. While NAT successfully connects private networks to the public Internet, it fundamentally breaks the end-to-end reachability model required for P2P protocols like WebRTC.

This document details why NAT complicates P2P communication, explains the underlying networking failures, and shows how protocols like STUN, ICE, and TURN work together to resolve these issues.

2 Scenario 1: Peer-to-Peer Without NAT (The Ideal Case)

To understand the problems introduced by NAT, consider an ideal network environment where every client device has a unique, globally routable **Public IP Address**.

Suppose two clients wish to connect:

- **Client 1:** Public IP address `49.32.100.8`
- **Client 2:** Public IP address `103.45.20.15`

Since both IP addresses are public, the global Internet routing system (routers, switches, and ISPs) knows exactly how to forward IP packets between these endpoints. Client 1 can construct a UDP packet with the destination set to `103.45.20.15` and send it. The packet is routed directly to Client 2. Similarly, Client 2 can respond directly to Client 1.

Figure 1: Scenario 1: Ideal Peer-to-Peer Connection without NAT

In this scenario, WebRTC session negotiation requires only the exchange of IP addresses and ports via a simple signaling server. Once the exchange completes, media flows directly between the clients without any issues.

3 Scenario 2: Peer-to-Peer with NAT (The Real-World Case)

In reality, home and corporate networks utilize private subnets. Devices behind a router do not have unique public IP addresses. Instead, they are assigned **Private IP Addresses** that are only valid within their local network.

Consider a real-world configuration:

- **User A (Local Client):**
  - Laptop IP (Private): `192.168.1.10`
  - Router A Public IP (WAN): `49.32.100.8`
- **User B (Remote Client):**
  - Laptop IP (Private): `192.168.0.5`
  - Router B Public IP (WAN): `103.45.20.15`

Figure 2: Scenario 2: Real-World Network Architecture with NAT Interfaces

Under this structure, if Laptop A tries to communicate directly with Laptop B using standard P2P methods, it faces several major problems.

4 The Three Core Problems of NAT in P2P

4.1 Problem 1: Non-Routable Private IP Addresses

The IP addresses assigned to laptops inside local networks belong to reserved private address spaces:

- `192.168.0.0` – `192.168.255.255` (Class C)
- `10.0.0.0` – `10.255.255.255` (Class A)
- `172.16.0.0` – `172.31.255.255` (Class B)

According to RFC 1918, these addresses are strictly non-routable on the public Internet. Global Internet routers are configured to drop any packet containing a private destination IP address.

If User B sends their local IP (`192.168.0.5`) in their WebRTC Session Description Protocol (SDP) offer, User A will attempt to send media packets to `192.168.0.5`. Since this is a private IP, User A's operating system or router will fail to route the packet to the WAN, or the packet will be dropped by the local ISP. Media transmission fails.

4.2 Problem 2: NAT Port-Binding and Incoming Packet Filtering

To bridge the gap between private and public IP networks, routers use a **NAT Mapping Table**.

When Laptop A sends an outgoing request (e.g., fetching a webpage at `192.0.2.1:80`), the router intercepts the packet. It replaces the source private IP and port with its own public WAN IP and a dynamically selected port. It then adds a mapping entry to its NAT table:

When the destination server sends a response back to `49.32.100.8:62000`, Router A checks its mapping table, sees that port `62000` corresponds to Laptop A (`192.168.1.10:5000`), modifies the destination header, and forwards the packet to the laptop.

Private Source IP	Private Port	→	Public Translated IP	Public Translated Port
192.168.1.10	5000	→	49.32.100.8	62000

Table 1: Example NAT Mapping Table Entry on Router A

4.2.1 The Unsolicited Incoming Packet Problem

Suppose Laptop A knows Laptop B's public IP address (`103.45.20.15`) and sends a packet directly to it on port `5000`. When the packet reaches Router B:

1. Router B receives a packet addressed to `103.45.20.15:5000`.
2. Router B checks its active NAT Mapping Table.
3. If Laptop B has not previously sent a packet *outbound* to Laptop A's public IP and port from its local port `5000`, no mapping entry exists in Router B's table.
4. Router B cannot determine which local device (`192.168.0.2`, `192.168.0.3`, or `192.168.0.5`) should receive the packet.
5. For security, Router B drops the packet as unsolicited incoming traffic.

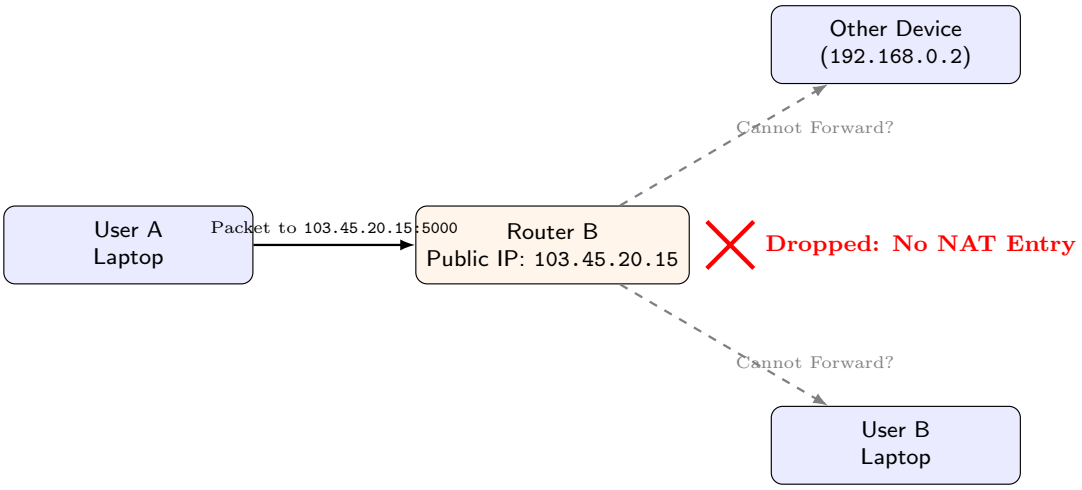

Figure 3: The Incoming Packet Forwarding Problem

4.3 Problem 3: Mutual Isolation (The Double-NAT Lockout)

If both User A and User B are behind separate NAT routers, they face a mutual lockout:

- User A cannot send a packet to User B's public IP because Router B has no mapping and will drop it.
- User B cannot send a packet to User A's public IP because Router A has no mapping and will drop it.
- Neither side can initiate the connection because the initial packet is dropped before a mapping table entry can be created on the remote router.

This scenario is known as the **NAT Traversal Problem**.

5 How WebRTC Resolves NAT: STUN, ICE, and TURN

To overcome these networking challenges, WebRTC relies on a suite of standard protocols: **STUN**, **ICE**, and **TURN**.

5.1 STUN (Session Traversal Utilities for NAT)

A STUN server is a public, lightweight utility hosted on the public Internet. It helps clients answer the question: *"What is my public IP address and port as seen by the outside world?"*

The discovery flow proceeds as follows:

1. Laptop A sends a UDP binding request to a public STUN server (e.g., `stun.l.google.com:19302`).
2. The packet passes through Router A. Router A translates the source address and creates an entry in its NAT table (e.g., mapping `192.168.1.10:5000` to `49.32.100.8:62000`).
3. The STUN server receives the packet from source `49.32.100.8:62000`.
4. The STUN server inspects the packet header, extracts the public IP and port, wraps them inside a STUN response payload, and sends it back.
5. Laptop A receives the STUN response and learns its public reflexive address: `49.32.100.8:62000`.

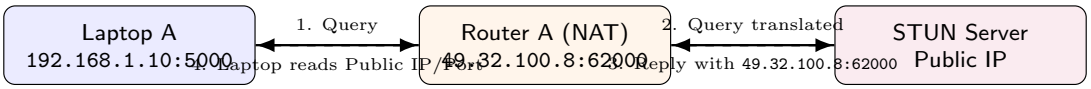

Figure 4: STUN Public IP Discovery Mechanism

5.2 ICE (Interactive Connectivity Establishment) Candidate Exchange

Once both clients discover their public addresses (known as **Server Reflexive Candidates**) alongside their local LAN addresses (**Host Candidates**), they must exchange these network options.

Because they cannot communicate directly yet, they exchange these candidates using an intermediate **Signaling Server** (typically running WebSockets or Socket.IO):

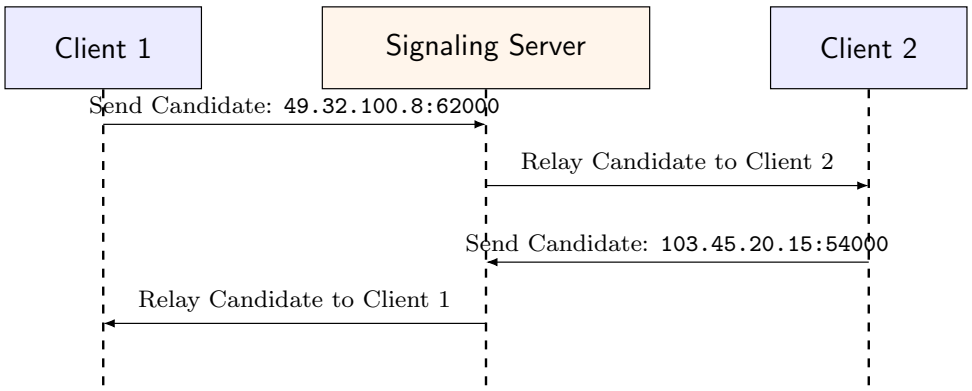

Figure 5: ICE Candidate Exchange sequence via Signaling Server

5.3 ICE Connectivity Checks (Hole Punching)

Once candidate exchange completes, both browsers perform **Connectivity Checks** by sending STUN binding packets directly to each other's candidate addresses.

This process enables **Hole Punching**:

1. Client 1 sends a packet to Client 2's public address (`103.45.20.15:54000`).
2. Router 1 intercepts the packet and creates an outbound NAT mapping entry (allowing incoming packets from `103.45.20.15:54000` to reach Client 1).
3. The packet arrives at Router 2. If Router 2 has not yet created a corresponding outbound mapping, it drops this first packet.
4. Concurrently, Client 2 sends a packet to Client 1's public address (`49.32.100.8:62000`).
5. Router 2 registers this outbound request, creating a mapping entry.
6. When this packet reaches Router 1, Router 1 recognizes the source address (`103.45.20.15:54000`) because of the mapping created in Step 2. Router 1 forwards the packet to Client 1.
7. Once Client 1 receives the packet, it sends a reply. Since Router 2 now has an active mapping entry for Client 1, the reply passes through.
8. The direct peer-to-peer path is established.

6 When P2P Fails: Symmetric NAT and TURN Servers

Direct hole punching works with most home routers (which use Full Cone, Restricted Cone, or Port Restricted Cone NAT). However, it fails when one or both networks use a strict NAT type, such as **Symmetric NAT**, commonly found in corporate networks, enterprise firewalls, and 3G/4G/5G mobile carriers.

6.1 Symmetric NAT Behavior

Under Symmetric NAT, the router does not reuse the same public port mapping for different destination IPs. If Client 1 sends packets from port `5000` to two different destination IPs, the router creates two distinct public port mappings:

- Outgoing to STUN Server → Mapped to `49.32.100.8:62000`
- Outgoing to Client 2 → Mapped to `49.32.100.8:63500`

Because the STUN server only observes port `62000`, Client 1 tells Client 2 to connect to port `62000`. However, when Client 2 attempts to send packets to `49.32.100.8:62000`, the symmetric router drops them because that mapping is restricted exclusively to traffic originating from the STUN server's IP address. Direct hole punching fails.

6.2 TURN (Traversal Using Relays around NAT)

When direct connection pathways fail, WebRTC falls back to a **TURN Server**. A TURN server acts as a media relay. Both clients establish connection legs to the TURN server, which forwards media packets between them.

While this approach guarantees that the call succeeds, it is no longer peer-to-peer:

- **Latency:** Media packets must travel through the TURN server, increasing latency.
- **Cost:** The service provider must pay for the server bandwidth consumed by the relayed media.
- **Scale:** The TURN server must scale horizontally to handle high media throughput.

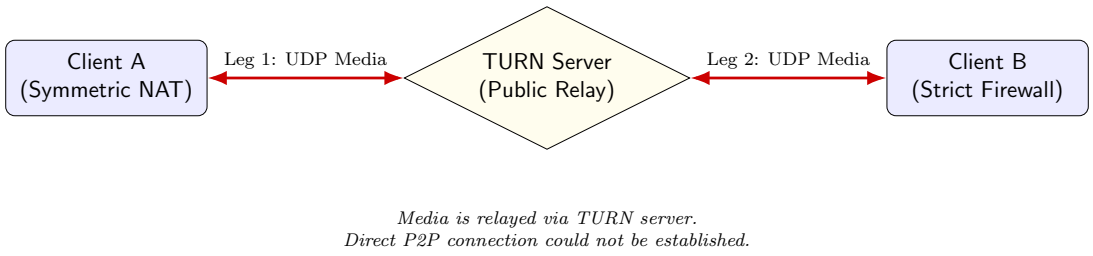

Figure 6: WebRTC Media Flow Relayed via a TURN Server

7 Summary: The WebRTC Connection Lifecycle

The following checklist summarizes the sequence of events during a WebRTC connection attempt:

1. **Candidate Gathering:** Both clients query local interfaces (Host Candidates) and STUN servers (Server Reflexive Candidates).

2. **SDP Exchange:** Clients exchange these candidates along with media descriptors (codecs, encryption keys) through the Signaling Server.

3. **Hole Punching:** Clients attempt direct UDP communication using the exchanged candidates.

4. **Fallback to TURN:** If direct paths fail due to Symmetric NAT or firewalls, the connection transitions to a relayed path via a TURN server.

This multi-tiered approach allows WebRTC to establish direct, low-latency P2P connections in over 85% of standard consumer network configurations, falling back to TURN relays only when necessary.